

EPAM Techno-Managerial Round Prep Guide

Your Project Deep-Dives · Microservices · Distributed Systems · NoSQL · HLD Case Studies
· LLD

Prepared for: Thursday, July 30, 2026 · 12:30-1:30 PM IST · 1 hr

1. What This Round Actually Tests

A: Based on documented EPAM techno-managerial rounds for 5+ YOE Java candidates, the structure typically runs: (1) Introduce yourself - skills, tech-stack, projects. (2) Explain your current project's architecture in detail, often drawn live on a whiteboard/draw.io. (3) A deep dive into each microservices component you mention - service registry internals, API gateway routing, config server, steps to add a new microservice, sync vs async communication, and distributed transaction patterns (2PC/Saga/CQRS) with drawbacks specifically probed. (4) Often, a lighter system-design or NoSQL/distributed-systems question to test breadth beyond your immediate project. (5) Team/delivery/estimation questions woven in throughout, tied to how you actually operated, not abstract theory.

» TIP: The single biggest lever in this round: narrate your OWN project fluently and specifically (real numbers, real trade-offs, real mistakes you caught) rather than reciting textbook definitions. Interviewers at this level are listening for ownership and judgment, not vocabulary.

How to structure your opening self-introduction (first 2-3 minutes)

A: 1) One line on total experience and domain breadth (Fintech + Healthcare, 6.5+ years). 2) Current role and what you own end-to-end (RDC reference-data governance platform at Macquarie/Synechron). 3) Tech stack in one breath (Java 17/21, Spring Boot, Kubernetes, PostgreSQL, ArgoCD). 4) One sentence on what kind of engineer you are (someone who takes ambiguous, messy problems - like a broken 130-column data model - and turns them into a governed, auditable system). Keep it under 90 seconds; let them pull threads with follow-up questions rather than front-loading everything.

2. Your Project Scenarios - STAR Bank (Macquarie / Synechron)

A: These are structured around what's already on your resume. Fill in the specific numbers/details you remember live, but the SHAPE of each story below is ready to use - lead with the technical problem, be honest about what was actually broken, and end with a measurable or qualitative outcome.

Scenario 1: The N+1 query issue

Situation: While working on the PMAP decommissioning application or the Instrument Browser's API layer over golden reference data, you noticed API response times degrading as related child data (e.g., pricing history, or related mapping rules) was being fetched.

Task: Diagnose why a seemingly simple list-fetch endpoint was slow, and fix it without breaking existing consumers.

Action: Identified that a JPA @OneToMany or lazy-loaded association was triggering one additional query per parent row returned (classic N+1). Fixed it with a JOIN FETCH in the JPQL query (or an @EntityGraph annotation), collapsing N+1 queries into a single join query. If Trino was involved, you may have also restructured the query to push the join down to the data plane rather than doing it in application code.

Result: State the actual latency improvement if you recall it (e.g., 'cut p95 latency from X to Y ms', or 'reduced DB round-trips from ~130 to 1 per request'). Even an approximate number is far more convincing than 'it got faster.'

```
// Before - lazy loading triggers N+1
List<Instrument> instruments = repository.findByExchange(exchange);
// each instrument.getPricingHistory() call = 1 extra query per instrument

// After - single query via JOIN FETCH
```

```
@Query("SELECT i FROM Instrument i JOIN FETCH i.pricingHistory WHERE i.exchange = :exchange")
List<Instrument> findByExchangeWithPricing(@Param("exchange") String exchange);
```

Scenario 2: The latency issue in the reference-data platform

Situation: The Instrument Browser needed to serve search, detail, and pricing history/sparkline data over Trino, sitting on top of a dual-data-source architecture (PostgreSQL control plane + Trino/Iceberg data plane).

Task: Trino queries over large Iceberg tables can be slow for interactive, user-facing latency expectations (sub-second for a UI sparkline) compared to what Trino is optimized for (large analytical scans).

Action: Likely approaches: pre-aggregating/pre-computing sparkline data instead of querying raw Iceberg data on every request; adding a caching layer (Redis or in-memory) in front of Trino for repeat/hot queries; partitioning/pruning Iceberg tables on commonly-filtered columns (e.g., date, instrument ID) so Trino scans less data per query; or moving the hot-path read data into the PostgreSQL control plane instead of hitting Trino for every request.

Result: Quantify the improvement (e.g., 'brought sparkline load time from several seconds to under 300ms with caching'), and mention the trade-off you accepted (e.g., cached data being seconds/minutes stale, acceptable for a sparkline but not for the golden record itself).

Scenario 3: The broken 130-column data model (Instrument Browser)

Situation: The Instrument Browser needed to resolve a broken 130-column data model with incorrect keys blocking correct retrieval of golden equity records.

Task: Untangle which of the 130 columns were actually meaningful join/lookup keys versus incorrectly modeled attributes, without a full rewrite that would block delivery.

Action: Root-caused which keys were wrong (e.g., a natural key like a ticker/ISIN being used where a surrogate key should have been used, or a many-to-many relationship modeled as one-to-one), redesigned the correct key structure, and built a migration/backfill path so existing consumers weren't broken mid-flight.

Result: Frame the outcome as: search/detail retrieval became correct and reliable, unblocking the consumer-facing API layer that depended on it.

Scenario 4: Data-quality remediation across ~75 canonical fields

Situation: You root-caused upstream ingestion gaps and schema drift affecting ~75 canonical fields feeding the reference-data governance platform.

Task: The initial ask was a vague 'the data quality is bad' complaint with no clear starting point.

Action: Systematically profiled the 75 fields against expected schema/business rules, categorized failures (missing values, type mismatches, drifted vendor formats from Bloomberg/LSEG), and converted the findings into a concrete, prioritized backlog ranked by business impact and effort - rather than trying to fix everything at once.

Result: This unblocked the team's mapping model and gave stakeholders a clear, trackable plan instead of an open-ended complaint. Good story for 'how do you turn ambiguity into an actionable plan.'

Scenario 5: SARK - Kubernetes Job manifests + real-time telemetry via Sidecar pattern

Situation: The SARK ecosystem needed dynamic workload orchestration with visibility into running container metrics.

Task: Programmatically generate Kubernetes Job manifests for different workload types, and get real-time metrics out of running containers without modifying every application's own code.

Action: Used the Kubernetes Java Fabric Client to generate Job manifests programmatically. Implemented three metrics-collection patterns: Sidecar (a companion container running alongside the main app container in the same Pod, sharing the same network namespace/volumes, scraping and forwarding metrics without touching the main app's code), File-based (main container writes metrics to a shared volume/file, a separate process reads and forwards them), and Database metrics job (a scheduled Job queries a DB directly for metrics rather than scraping a running process).

Result: Gave the SARK ecosystem dynamic, pattern-based observability across heterogeneous workloads without hardcoding metrics logic into every job type - reusable orchestration.

```
# Simplified idea of a Sidecar pattern Pod spec (what you'd describe verbally)
apiVersion: v1
kind: Pod
spec:
  containers:
    - name: main-app
      image: sark-worker:latest
    - name: metrics-sidecar
      image: metrics-forwarder:latest
      # shares the Pod's network + can mount the same volume as main-app
      volumeMounts:
        - name: shared-metrics
          mountPath: /var/metrics
  volumes:
    - name: shared-metrics
      emptyDir: {}
```

Scenario 6: Sybase to PostgreSQL migration across 3 microservices

Situation: PMAP, Importer, and Exporter services all needed to move off Sybase onto PostgreSQL, as sole or primary backend owner for at least PMAP.

Task: Migrate the schema and data without breaking any of the three services' contracts, across DEV/UAT/PROD, over a 4-month engagement.

Action: Likely involved: mapping Sybase-specific data types/syntax to PostgreSQL equivalents, writing migration scripts, running a parallel-run or phased cutover per environment, and full regression/integration testing per service before promoting to the next environment.

Result: Completed full SDLC and production releases for all three services - a strong story for 'tell me about a large, multi-service change you owned end-to-end.'

Scenario 7: SOLID refactor of MCP SARK, reaching ~90% test coverage

Situation: MCP SARK application had maintainability/reusability issues.

Action: Applied SOLID principles explicitly - likely extracting responsibilities into smaller, single-purpose classes (SRP), introducing interfaces so implementations could vary without touching callers (DIP/OCP), and restructuring inheritance hierarchies that were violating LSP.

Result: Improved reusability/maintainability and increased unit test coverage to ~90%, reducing defects and improving system stability - good story for 'how do you improve a legacy codebase without a full rewrite.'

Scenario 8: TCS - NDM to HCP ingestion, 90% transfer time reduction

Situation: A Spring Boot microservice needed to ingest records from NDM to HCP.

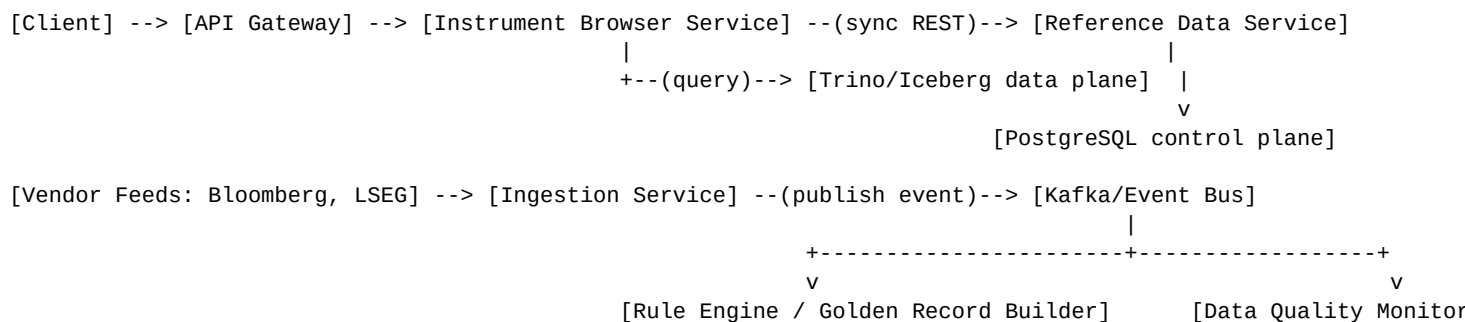
Action: Optimized real-time file movement - likely batching, parallelizing transfers, or eliminating an unnecessary intermediate step/serialization overhead in the pipeline.

Result: 90% reduction in transfer time, with ~100% test coverage on the service - strong, quantified performance story.

3. Microservices Deep Dive - As Asked Live in Real EPAM Rounds

Q: Draw and explain your current project's microservices architecture.

A: Practice this on an actual whiteboard/draw.io before the call. Structure: boxes for each service, arrows labeled sync (REST/Feign) or async (Kafka/event), the data stores each service owns, and the API Gateway/Service Registry sitting at the edge. Narrate it top-down: entry point -> routing -> core services -> data layer -> async/event flows.



Q: Service Registry - how does it work, and what is a heartbeat message?

A: Each service instance registers itself with the registry (e.g., Eureka) on startup, sending its host, port, and instance ID. After registering, it periodically sends a heartbeat (a lightweight 'I'm still alive' ping, typically every ~30 seconds by default in Eureka) to the registry. If the registry misses a configurable number of consecutive heartbeats (default: 3 missed => ~90 seconds), it marks that instance as DOWN and stops routing traffic to it. Heartbeat interval and lease-expiration duration are typically configured via `eureka.instance.lease-renewal-interval-in-seconds` and `lease-expiration-duration-in-seconds`.

Q: API Gateway - deep dive into internal routing. How does it route a request to a specific instance of a service?

A: The gateway receives the request, extracts the target service name from the route configuration (matched by path prefix, e.g., `/api/orders/**` -> `order-service`), then asks the Service Registry (or its locally cached registry snapshot) for the list of healthy instances currently registered under that service name. It then picks one instance using a client-side load-balancing strategy (round-robin is Spring Cloud LoadBalancer's default), rewrites/forwards the request to that instance's actual host:port, and streams the response back to the original caller. The gateway itself typically also handles cross-cutting filters here - auth token validation, rate limiting, request logging - before or after the routing decision.

Q: Config Server - what configuration is stored, and how do services fetch it?

A: It stores externalized configuration (usually in a Git-backed repo) organized per application name and per profile/environment - e.g., `order-service-prod.yml`, `order-service-dev.yml`, plus a shared `application.yml` for common config. On startup, each service (configured with `spring.config.import=configserver:` or the older `bootstrap.yml` approach) calls the Config Server's REST endpoint (`/application/{profile}`) to fetch its configuration before the rest of its Spring context initializes, so config values are available early enough to configure `DataSources`, feature flags, etc. Runtime config changes require a POST to `/actuator/refresh` (or `/actuator/bus-refresh` for fleet-wide) on the consuming service to re-pull and apply changes without a restart.

Q: Walk through, step by step, what changes are needed to introduce a brand-new microservice into your current architecture.

A: 1) Define the service's bounded context and API contract (what data/operations it owns, what it exposes). 2) Scaffold the Spring Boot project with the standard layering (Controller/Service/Repository) and its own dedicated database/schema - never share a database directly with another service. 3) Register it with the Config Server (add its config file to the Git repo) and Service Registry (add spring-cloud-starter-netflix-eureka-client dependency + eureka.client.service-url config). 4) Add routing rules in the API Gateway for its path prefix. 5) Decide sync vs async integration with existing services - add a FeignClient interface if sync, or a Kafka producer/consumer if async/event-driven. 6) Add it to the CI/CD pipeline (build, test, containerize, deploy stages) and to centralized logging/monitoring (correlation ID propagation, health check endpoint wired into the orchestrator). 7) Update the circuit breaker/resilience config for any new synchronous calls made to or from it.

Q: Sync vs async communication - RestTemplate/FeignClient in practice, and when you'd choose each.

A: Use synchronous REST (FeignClient preferred for its declarative simplicity and registry/load-balancer integration) when the caller genuinely needs an immediate response to proceed - e.g., checking real-time inventory before confirming an order. Use asynchronous messaging (Kafka) when the caller doesn't need to wait - e.g., 'order placed' can fire an event that a notification service consumes whenever it gets to it, without blocking the order flow. In a reference-data platform, ingestion from vendor feeds (Bloomberg/LSEG) is a natural async fit - you don't want ingestion blocked waiting for every downstream consumer.

Q: 2PC vs Saga vs CQRS - practical implementation questions, and Saga's drawbacks specifically.

A: If your platform ever needed to keep two data stores in sync (e.g., the PostgreSQL control plane and the Trino/Iceberg data plane) as part of one logical operation, that's a natural place to discuss Saga: publish an event after the control-plane write commits, have a downstream consumer apply the corresponding change to the data plane, and if that downstream step fails, run a compensating action (e.g., mark the control-plane record as 'pending-sync-failed' rather than trying to roll back a already-committed transaction). Saga's real drawback you can speak to from experience: debugging a partially-completed multi-step flow is harder than debugging a single transaction - you need good correlation-ID-based logging to reconstruct what actually happened across services when a step fails partway through.

4. Distributed Systems Fundamentals (Gaurav Sen Core Topics)

Q: Explain the CAP theorem, and give a real-world example of a system choosing CP vs AP.

A: In a distributed system, when a network Partition occurs (nodes can't communicate), you must choose between Consistency (every read gets the most recent write, but the system may refuse requests it can't guarantee are consistent) and Availability (every request gets a response, but it might not be the latest data). You can't have both fully during a partition - hence 'pick 2 of 3,' though in practice Partition tolerance is mandatory for any real distributed system, so the real choice is CP vs AP. Example: a banking ledger balance check typically favors CP (would rather return an error than show a stale balance). A social media 'like count' or product view count typically favors AP (better to show a slightly stale number than fail the whole page load).

Q: What is PACELC, and how does it extend CAP theorem?

A: PACELC says: if there's a Partition (P), you choose between Availability (A) and Consistency (C) - that's standard CAP. But Else (E, i.e., when there's no partition, the normal case), you still have to choose between Latency (L) and Consistency (C), because even without a partition, achieving strong consistency across replicas (waiting for all/majority to acknowledge a write) costs latency. This is why PACELC is considered a

more complete, practically useful framework than CAP alone - CAP only talks about the rare partition scenario, but the latency/consistency trade-off exists all the time.

Q: What is consistent hashing, and what problem does it solve over simple modulo-based hashing?

A: With simple modulo hashing ($\text{server} = \text{hash}(\text{key}) \% N$), adding or removing a single server changes the modulo for almost every key, forcing a massive redistribution/cache-miss storm across the whole cluster. Consistent hashing places both servers and keys onto a conceptual ring (via a hash function), and each key is assigned to the next server clockwise on the ring. Adding/removing one server only affects the keys between it and its immediate neighbor on the ring - a small, localized fraction of keys - not the whole keyspace. Virtual nodes (each physical server mapped to multiple points on the ring) are used to keep the key distribution balanced even with a small number of servers.

```
// Conceptual consistent hashing ring lookup
FUNCTION getServerForKey(key):
    hash = hashFunction(key)
    // find the first server clock-wise on the ring whose hash >= this key's hash
    RETURN ring.ceilingEntry(hash) OR ring.firstEntry() // wrap around if past the last node
```

Q: What load balancing algorithms exist, and when would you pick one over another?

A: Round Robin - simplest, cycles through servers evenly; good when all requests/servers are roughly equal cost. Least Connections - routes to the server currently handling the fewest active requests; better when request processing time varies significantly. Weighted Round Robin - like round robin but some servers get proportionally more traffic (useful for heterogeneous hardware). IP Hash / Consistent Hashing - routes based on a hash of the client (or key), useful when you need the same client to consistently hit the same server (session affinity, or cache locality).

Q: Explain database replication: leader-follower vs multi-leader, and synchronous vs asynchronous replication.

A: Leader-follower (most common): all writes go to a single leader, which then propagates changes to followers; followers typically serve read traffic to scale reads. Multi-leader: multiple nodes can accept writes independently (useful for multi-region setups), at the cost of needing conflict resolution when the same data is written differently in two places concurrently. Synchronous replication waits for follower(s) to acknowledge before confirming the write to the client (safer, but higher latency and the write blocks if a follower is slow/down). Asynchronous replication confirms the write immediately after the leader persists it, propagating to followers in the background (faster, but a leader crash before propagation means recent writes can be lost).

Q: What is sharding/partitioning, and what are common partitioning strategies?

A: Sharding splits a large dataset across multiple database nodes so no single node needs to hold or serve all the data. Range-based partitioning splits by a key range (e.g., user IDs 1-1M on shard A, 1M-2M on shard B) - simple but can create hot shards if traffic isn't evenly distributed across ranges. Hash-based partitioning applies a hash function to the key to decide the shard - more even distribution, but range queries become expensive since related keys are scattered. Directory-based partitioning uses a lookup service to map keys to shards - flexible/rebalanceable, but that lookup service becomes a critical dependency.

Q: What is a CDN, and why does it help even for dynamic-feeling applications?

A: A Content Delivery Network caches content at edge servers geographically close to users, reducing latency by avoiding a round-trip to the origin server for every request. Even for apps that feel 'dynamic,' large static assets (JS bundles, images, CSS) and semi-static API responses (e.g., product catalogs, reference data that changes rarely) benefit hugely from CDN caching, offloading significant traffic away from origin servers.

Q: What is a distributed lock, and when would you need one (e.g., Redis-based)?

A: A distributed lock ensures only one process across multiple service instances can perform a critical action at a time - e.g., only one instance of a scheduled job should run a specific data-migration task, even if the job is deployed across 5 pods. Commonly implemented via Redis (SET key value NX EX ttl - set a key only if it doesn't already exist, with an expiry so a crashed holder doesn't lock others out forever) or via a distributed coordination service like Zookeeper/etcd.

Q: What is leader election, and why does a distributed system need it?

A: In a cluster of otherwise equal nodes, leader election designates one node to make certain centralized decisions (e.g., which node processes a particular partition, or coordinates a distributed transaction), avoiding conflicting concurrent decisions. Algorithms like Raft or Paxos (or simpler tools like Zookeeper's ZAB protocol) ensure the cluster agrees on exactly one leader even amid node failures or network issues, and can elect a new leader automatically if the current one goes down.

5. NoSQL Databases

Q: What are the main types of NoSQL databases, and give an example/use case for each?

A: Key-Value stores (Redis, DynamoDB) - simplest model, O(1) lookups by key, great for caching, session storage, feature flags. Document stores (MongoDB) - store semi-structured JSON-like documents, good when your schema varies across records or nests naturally (e.g., a product catalog with varying attributes per category). Column-family stores (Cassandra, HBase) - optimized for very high write throughput and wide, sparse rows, commonly used for time-series or event-logging data at massive scale. Graph databases (Neo4j) - optimized for traversing relationships (friend-of-friend, fraud-ring detection, recommendation engines) where the relationships themselves are the primary query pattern, not just the entities.

Q: What is BASE, and how does it differ from ACID?

A: BASE (Basically Available, Soft state, Eventually consistent) is the consistency model most NoSQL databases favor, prioritizing availability and partition tolerance over strict consistency. Basically Available - the system guarantees a response, even if it's not the absolute latest data. Soft state - data may change over time even without new writes, as replicas converge. Eventually consistent - given enough time without new writes, all replicas will converge to the same value. This contrasts with ACID's strict guarantee that a transaction is immediately, fully consistent everywhere the moment it commits.

Q: When would you choose a NoSQL database over a relational one, concretely?

A: When your access pattern is simple key-based lookups at very high scale/throughput (session data, caching). When your schema is genuinely variable/evolving across records and forcing a rigid relational schema would mean constant migrations. When you need horizontal write scalability beyond what a single relational primary can handle, and you can tolerate eventual consistency for that data. Conversely, stick with relational/SQL when you need strong transactional guarantees across multiple related entities (e.g., financial ledger entries, which is likely why your Macquarie platform's control plane uses PostgreSQL, not a NoSQL store, for governance data).

Q: How would you explain choosing Postgres (control plane) + Trino/Iceberg (data plane) as a dual-store architecture, in NoSQL/polyglot-persistence terms?

A: This is a form of polyglot persistence - using the right storage engine for each access pattern rather than one database for everything. PostgreSQL as the control plane fits because governance/rule metadata needs strong ACID guarantees, referential integrity, and transactional consistency (you can't have half-applied governance rules). Trino/Iceberg as the data plane fits because it's built for large-scale analytical scans over semi-structured, high-volume reference data (vendor feeds), where you need to query huge datasets efficiently but don't need per-row transactional guarantees the way you do for the rules themselves.

6. High-Level Design Case Studies (Gaurav Sen-Style)

» TIP: For each of these, narrate in this order out loud: (1) clarify requirements/scale, (2) high-level components, (3) data model, (4) how you'd handle the one hardest part (usually scale or consistency), (5) trade-offs. Interviewers care far more about your reasoning process than a 'correct' final diagram.

Q: Design a URL Shortener (TinyURL).

A: Requirements: given a long URL, generate a short unique code; redirect short URL to original on access; handle high read (redirect) to write (create) ratio. Core approach: generate a short code either via a base62 encoding of an auto-incrementing ID (guarantees uniqueness, simple), or a hash of the URL truncated to a fixed length (risk of collisions, needs a retry/salt strategy). Store a mapping table (short_code -> long_url, created_at, expiry) in a key-value or relational store - reads dominate massively, so this benefits heavily from a caching layer (Redis) in front of the DB for hot/recently-created links. For scale, shard the mapping table by short_code hash across multiple DB nodes, and use a CDN/edge cache for extremely popular redirects.

```
[Client] --> [Load Balancer] --> [App Servers] --> [Redis Cache] --> [Sharded DB: code -> long_url]
                                     |
                                     (cache miss falls through to DB, then populates cache)
```

Q: Design a Parking Lot system (HLD + LLD, a very common one).

A: HLD requirements: multiple entry/exit points, multiple vehicle types (car/bike/truck) needing different spot sizes, track availability in real time, compute a parking fee on exit. HLD components: an Entry/Exit Gate service that assigns/frees a spot, a Spot Availability service (often backed by a fast in-memory store like Redis for real-time counts per floor/spot-type), and a Payment/Billing service computing fee based on duration and vehicle type.

```
// LLD - core class design
abstract class Vehicle {
    String licensePlate;
    VehicleType type; // CAR, BIKE, TRUCK
}

abstract class ParkingSpot {
    String spotId;
    boolean isOccupied;
    abstract boolean canFitVehicle(Vehicle v);
}

class CompactSpot extends ParkingSpot {
    boolean canFitVehicle(Vehicle v) { return v.type == VehicleType.CAR || v.type == VehicleType.BIKE; }
}

class ParkingFloor {
    List<ParkingSpot> spots;
    Optional<ParkingSpot> findAvailableSpot(Vehicle v) {
        return spots.stream()
            .filter(s -> !s.isOccupied && s.canFitVehicle(v))
            .findFirst();
    }
}

class ParkingLot {
    List<ParkingFloor> floors;
    Map<String, Ticket> activeTickets; // licensePlate -> Ticket

    Ticket parkVehicle(Vehicle v) {
        for (ParkingFloor floor : floors) {
            Optional<ParkingSpot> spot = floor.findAvailableSpot(v);
            if (spot.isPresent()) {
```

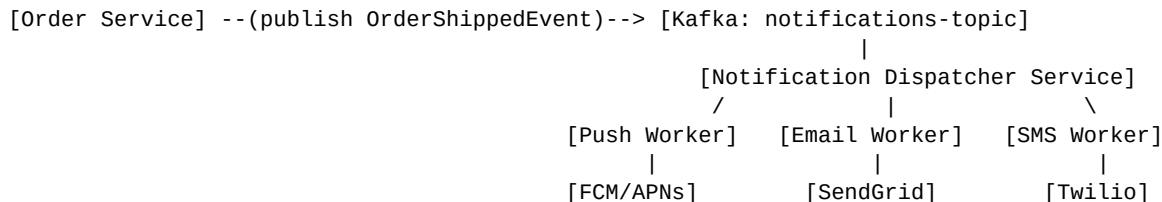
```

        spot.get().isOccupied = true;
        Ticket ticket = new Ticket(v, spot.get(), Instant.now());
        activeTickets.put(v.licensePlate, ticket);
        return ticket;
    }
}
throw new IllegalStateException("Parking lot full");
}
}

```

Q: Design a Notification System (push/email/SMS across many services).

A: Requirements: multiple services need to trigger notifications; multiple channels (push, email, SMS); must not block the triggering service; must handle retries/failures per channel independently. HLD: any service publishes a NotificationRequested event to a message queue (Kafka) instead of calling a notification API synchronously. A Notification Service consumes these events, determines the user's channel preferences, and fans out to channel-specific workers (Push Worker, Email Worker, SMS Worker), each with its own retry/backoff logic and third-party provider integration (FCM/APNs, SendGrid, Twilio). A Notification Log/Status store tracks delivery status per notification for auditing and debugging.



Q: Design WhatsApp/a chat system (high level).

A: Core challenges: real-time message delivery, message ordering, delivery/read receipts, offline users, group chats, media sharing. HLD: clients maintain a persistent WebSocket connection to a Chat Gateway service; each gateway instance tracks which users are connected to it (in a distributed session registry, e.g., Redis). Sending a message: client -> gateway -> Message Service, which persists the message (per-conversation partition in a database like Cassandra, well-suited for high write throughput and append-heavy chat history) and looks up the recipient's connected gateway instance to push it in real time; if the recipient is offline, the message waits in their queue/inbox until they reconnect, at which point missed messages are synced. Read/delivery receipts are separate lightweight events flowing the same path.

Q: Design a Rate Limiter (system-level, not just the earlier single-instance Java code).

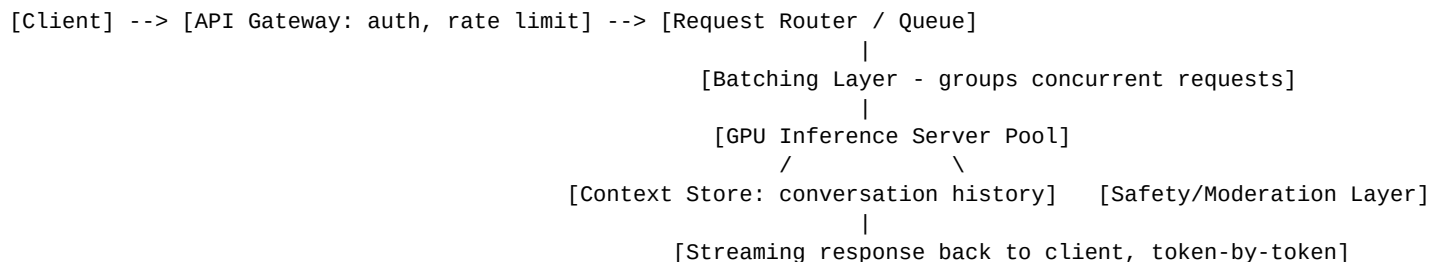
A: At scale, a rate limiter needs to work correctly across many API server instances, not just in one process's memory - so counters must live in a shared, fast store (Redis), not in-memory per instance. Common algorithms: Fixed Window (simple, but allows bursts at window boundaries), Sliding Window Log (accurate but memory-heavy - stores every request timestamp), Sliding Window Counter (approximates sliding window cheaply by weighting the previous and current fixed windows), and Token Bucket (allows controlled bursts up to a bucket capacity while maintaining a steady refill rate - often the preferred choice for APIs wanting to allow occasional bursts).

Q: Design a News Feed system (e.g., Instagram/Twitter feed).

A: Core tension: Push (fan-out-on-write) vs Pull (fan-out-on-read). Push model: when a user posts, immediately write that post into every follower's precomputed feed (fast reads, but expensive/slow writes for users with millions of followers - the classic 'celebrity problem'). Pull model: feed is computed at read time by merging recent posts from everyone a user follows (cheap writes, but expensive reads, especially for users following many people). Most real systems use a hybrid: push for typical users, pull (or a separate path) for high-follower-count accounts, merging both at read time.

Q: Design a high-level architecture for an LLM-serving system (like Claude/ChatGPT).

A: Requirements: handle massive concurrent chat requests, maintain conversation context, stream tokens back in real time, and route to the right underlying model efficiently. HLD: an API Gateway handles auth/rate-limiting, then routes to a Request Router that picks an available GPU-backed Inference Server (often behind a queue, since inference is expensive and needs batching for GPU efficiency - request batching groups multiple concurrent requests together to maximize GPU utilization). A Conversation/Context Store (likely a fast key-value or document store) holds recent conversation history so context doesn't need to be resent by the client on every turn. Responses are streamed back to the client token-by-token over a persistent connection (SSE or WebSocket) rather than waiting for the full response, to keep perceived latency low. A separate Safety/Moderation layer may screen both input and streaming output. At Claude's/ChatGPT's scale, this also needs careful GPU fleet autoscaling, model-version routing (which checkpoint serves which request), and aggressive caching of common prefixes/system prompts to avoid redundant computation.



7. Techno-Managerial Soft-Skill Layer

Q: How do you estimate a task you've never done before (e.g., migrating Sybase to PostgreSQL for the first time)?

A: Break it into known sub-tasks and flag the genuinely unknown ones separately rather than giving one number for the whole thing. For unknowns, do a short time-boxed spike/investigation first, then re-estimate with that information. Always attach a confidence level or range (e.g., '2-3 weeks, but flagging the data-type mapping step as the biggest unknown') rather than a false-precision single number, and communicate that uncertainty to stakeholders upfront rather than after you miss the date.

Q: Tell me about a time you had to push back on a requirement or timeline you thought was wrong.

A: Strong shape: describe the specific ask, why it seemed problematic technically (not just 'I didn't want to'), how you brought data/reasoning to the conversation rather than just refusing, and what the actual resolution was - even if you didn't get everything you wanted, showing you engaged constructively rather than either silently complying or being obstinate is the signal they're looking for.

Q: How do you handle a production incident under pressure, especially with a client watching closely (relevant given Macquarite is a client engagement)?

A: Stabilize first, root-cause second - i.e., mitigate customer impact (rollback, feature flag off, scale up) before doing a deep investigation, unless the fix is already obvious. Communicate proactively and on a predictable cadence to the client/stakeholders even when you don't have a full answer yet ('here's what we know, here's what we're doing next, here's when we'll update you again') rather than going quiet until it's fully resolved. Afterward, do a blameless postmortem focused on the systemic/process gap, not individual fault.

Q: How do you decide when to pay down technical debt versus keep shipping features?

A: Frame technical debt in terms of its ongoing cost - is it actively slowing down delivery velocity, causing repeated production incidents, or blocking a specific upcoming feature? If yes, it earns priority alongside feature work, ideally quantified (e.g., 'this area caused 3 of our last 5 incidents'). If it's just aesthetically imperfect code that isn't causing measurable pain, it's reasonable to defer it - not all debt is worth paying down immediately, and saying so shows pragmatism rather than perfectionism.

Q: How do you mentor or unblock a junior team member who's stuck?

A: Ask what they've already tried before jumping to the answer, to understand their actual gap and avoid just doing the thinking for them. Time-box how long you let them stay stuck before stepping in, balancing learning opportunity against delivery risk. Prefer pointing them toward the right question/resource over handing over a finished solution, but don't let pride in 'making them figure it out' cause a missed deadline - know when to just show them directly.

Q: How do you communicate a technical delay or a broken data model issue (like the 130-column problem) to a non-technical stakeholder or client?

A: Translate the technical issue into business impact in one sentence first ('search results are returning wrong records because of how the underlying data is structured'), state the plan and realistic timeline, and avoid over-explaining the technical root cause unless asked - lead with impact and plan, have the technical depth ready if they want it.

8. Multithreaded Systems, Async Processing & Kafka/Queues

Q: Walk through how you'd design a multithreaded worker pool to process items from a queue.

A: Use an ExecutorService-backed thread pool where each worker thread pulls a task, processes it, and goes back for more - rather than spinning up a thread per task, which doesn't scale. Size the pool based on whether work is CPU-bound (roughly number of cores) or I/O-bound (can be much higher, since threads spend most time waiting, not computing). Use a bounded BlockingQueue as the work queue so producers don't overwhelm the system - if the queue is full, apply a backpressure policy (block the producer, drop the task, or reject with a RejectedExecutionHandler).

```
ExecutorService pool = new ThreadPoolExecutor(  
    10, // core pool size  
    20, // max pool size  
    60L, TimeUnit.SECONDS, // idle thread keep-alive  
    new LinkedBlockingQueue<>(500), // bounded work queue - backpressure point  
    new ThreadPoolExecutor.CallerRunsPolicy() // if queue is full, caller thread runs it (natural throttling)  
);  
  
pool.submit(() -> processOrder(order));
```

Q: What is the difference between a producer-consumer pattern using BlockingQueue vs using Kafka?

A: A BlockingQueue is in-memory, within a single JVM - fast, but data is lost if the process crashes, and it doesn't scale across multiple service instances. Kafka is a distributed, persistent log - producers and consumers can be entirely separate services/instances, messages survive a consumer crash (they stay in the topic until acknowledged/committed), and multiple consumer instances can share the load via consumer groups. Use BlockingQueue for in-process task handoff; use Kafka when producers and consumers are different services, need durability, or need to scale independently.

Q: Explain Kafka's core concepts: topic, partition, consumer group, offset.

A: A topic is a named stream of messages (e.g., 'order-events'). Each topic is split into partitions, which allow parallelism - each partition is an ordered, append-only log, but ordering is only guaranteed within a single partition, not across the whole topic. A consumer group is a set of consumer instances sharing the work of reading a topic - Kafka assigns each partition to exactly one consumer within a group, so a topic with 4 partitions can be processed in parallel by up to 4 consumer instances in the same group. The offset is a per-partition pointer tracking how far a consumer group has read - committing an offset marks messages as

processed, so a restarted consumer resumes from there instead of reprocessing everything.

```
@KafkaListener(topics = "order-events", groupId = "notification-service")
public void onOrderEvent(OrderEvent event, Acknowledgment ack) {
    try {
        notificationService.notify(event);
        ack.acknowledge(); // manually commit offset only after successful processing
    } catch (Exception e) {
        // don't acknowledge - message will be redelivered on retry/rebalance
        log.error("Failed to process event, will retry", e);
    }
}
```

Q: How do you guarantee message ordering in Kafka when you need it (e.g., all events for one order must be processed in order)?

A: Use the entity's ID (e.g., orderId) as the message key. Kafka guarantees all messages with the same key land in the same partition, and within a single partition, order is strictly preserved. So as long as your key choice keeps related events together, ordering is guaranteed for that entity even though the topic as a whole is processed in parallel across partitions.

Q: What is the difference between at-most-once, at-least-once, and exactly-once delivery semantics in Kafka?

A: At-most-once: offset is committed before processing - if the consumer crashes mid-processing, that message is lost, never reprocessed. At-least-once (most common default): offset is committed after successful processing - if the consumer crashes after processing but before committing, the message is redelivered and processed again, so consumers must be idempotent. Exactly-once: achieved via Kafka's transactional producer/consumer APIs, ensuring a message is processed and any resulting writes/offset commits happen atomically - more complex to set up and with some throughput cost, used when duplicate processing is unacceptable (e.g., financial transactions).

Q: Why must Kafka consumers generally be designed to be idempotent?

A: Because at-least-once delivery (the common default) can redeliver a message after a crash or rebalance, even though it was already fully or partially processed once. If processing that message has a side effect (like charging a payment or sending an email), reprocessing it without idempotency causes duplicate charges/emails. Idempotency means processing the same message twice produces the same end state as processing it once.

Q: What is Spring WebFlux, and when would you choose it over traditional Spring MVC?

A: Spring MVC uses a thread-per-request model - each incoming request occupies a thread for its entire duration, including while blocked waiting on I/O (a DB call, a downstream API call). WebFlux is built on Project Reactor (reactive streams) and uses a small, fixed number of event-loop threads that are never blocked waiting on I/O - when a request needs to wait for a DB/network response, the thread is freed to handle other requests, and resumes this request when the response arrives. This lets a WebFlux app handle far more concurrent connections with far fewer threads, which matters most for I/O-heavy workloads with high concurrency (e.g., a gateway proxying many downstream calls). For CPU-bound or straightforward CRUD workloads with modest concurrency, traditional Spring MVC is simpler to write, debug, and reason about, and the added complexity of reactive code (Mono/Flux chains, harder stack traces) often isn't worth it.

```
// Spring MVC - blocking, thread-per-request
@GetMapping("/orders/{id}")
public Order getOrder(@PathVariable Long id) {
    return orderRepository.findById(id); // thread blocks here until DB responds
}

// Spring WebFlux - non-blocking, reactive
@GetMapping("/orders/{id}")
```

```
public Mono<Order> getOrder(@PathVariable Long id) {
    return orderRepository.findById(id); // returns immediately with a Mono; thread is freed while DB responds
}
```

» TIP: A very fair, honest answer if pressed on WebFlux depth: 'I understand the thread-per-request vs event-loop trade-off and when each fits, but most of my hands-on production work has been on traditional Spring MVC with async offloaded to Kafka rather than reactive streams' - that's a credible, specific answer, not evasive.

9. Advanced Microservices & Distributed Systems

Q: REST/gRPC vs Message Brokers (Kafka/RabbitMQ) - how do you actually decide?

A: Choose synchronous REST/gRPC when the caller needs an immediate answer to proceed (e.g., 'is this item in stock' before confirming checkout), and the two services can tolerate being coupled in time (both must be up simultaneously). Choose gRPC specifically over REST when you need lower latency and strongly-typed contracts between internal services (binary protobuf vs JSON, HTTP/2 multiplexing) - common for internal service-to-service calls, less common for public-facing APIs where REST/JSON's ubiquity and human-readability matter more. Choose a message broker (Kafka for high-throughput event streams and replayable logs; RabbitMQ for more traditional task-queue semantics with flexible routing) when the producer shouldn't be blocked waiting, when multiple independent consumers need the same event, or when you want to decouple services in time so one being temporarily down doesn't block the other.

Q: Distributed tracing: a request fails or slows down across a chain of 5 microservices. How do you actually track it down?

A: Every request gets a unique trace ID generated at the edge (API Gateway), and each service propagates that trace ID (plus a span ID identifying its own piece of work) forward - via HTTP headers for synchronous calls, and via message headers/metadata for Kafka events. Micrometer Tracing (the modern replacement for Spring Cloud Sleuth) auto-instruments this propagation in Spring Boot apps, and exports the trace data to a backend like Zipkin or Jaeger (often via the OpenTelemetry protocol), which stitches together every span across all 5 services into a single visual timeline. You can then see exactly which service in the chain took the longest, or where the failure actually originated, instead of grepping through 5 separate services' logs independently.

```
// Trace ID/span propagation - handled largely automatically by Micrometer Tracing,
// but conceptually every hop passes this along:
// HTTP header: traceparent: 00-<trace-id>-<span-id>-01
// Kafka message header: traceparent added to ProducerRecord headers before send,
// read back out by the consumer to continue the same trace
```

Q: API Gateway edge patterns: centralized vs decentralized authentication - what's the trade-off?

A: Centralized auth (validate the JWT once at the API Gateway, then trust internal service-to-service calls, or pass a lightweight internal token/claims forward) reduces duplicate validation logic and keeps a single enforcement point, but makes the gateway a critical dependency and single point of policy control. Decentralized auth (every service independently validates the JWT/token on every call) is more resilient to a single point of failure and lets each service enforce its own fine-grained authorization rules, at the cost of duplicated validation logic and slightly more latency per hop. Most real systems land on a hybrid: gateway does coarse-grained auth (is this a valid, non-expired token at all) and individual services still do their own fine-grained authorization (can this specific user perform this specific action).

Q: How would you configure Spring Cloud Gateway for rate limiting and JWT validation at the edge?

```
# application.yml - Spring Cloud Gateway route with Redis-based rate limiting
spring:
  cloud:
```



```

gateway:
  routes:
    - id: order-service-route
      uri: lb://order-service
      predicates:
        - Path=/api/orders/**
      filters:
        - name: RequestRateLimiter
          args:
            redis-rate-limiter.replenishRate: 10    # steady-state requests/sec allowed
            redis-rate-limiter.burstCapacity: 20    # short burst allowance above the steady rate
        - name: TokenRelay    # forwards the validated JWT downstream

```

A: The RedisRateLimiter here implements a token-bucket algorithm backed by Redis, so rate limits are enforced consistently across all gateway instances (not just per-instance in memory). JWT validation typically happens via a global filter that checks the token's signature and expiry before the request is even routed, rejecting invalid tokens with a 401 before they reach any downstream service.

10. Deep Dive: Distributed Transactions & Resilience

Q: Saga: Choreography vs Orchestration - explain both, with the practical trade-off.

A: Choreography (event-driven): each service publishes an event when it completes its step, and the next service in the flow simply listens for that event and reacts - there's no central coordinator, services are only coupled through the events themselves. Simple for a short flow, but as the number of steps grows, it becomes hard to see the overall flow just by reading code (the 'flow' is implicit, scattered across many services' event listeners), making debugging and onboarding harder. Orchestration (command-driven): a central orchestrator explicitly tells each service what to do next, and directly handles failure/compensation logic in one place. Easier to understand and monitor the end-to-end flow (it's explicit code in one location), at the cost of that orchestrator becoming a critical, more complex component itself.

Q: How would you implement Saga orchestration in a Java/Spring stack - what tools, concretely?

A: Two common paths: (1) A workflow engine like Camunda or Temporal, where you define the saga as an explicit workflow/process definition, and the engine handles state persistence, retries, and compensation logic for you - good for complex, long-running, or highly auditable business processes. (2) A lighter, custom approach using Spring Cloud Stream with Kafka, where your own orchestrator service publishes commands to specific topics per step, listens for the corresponding success/failure events, and manually implements the compensation logic in code - more control, more work, suited to simpler flows where a full workflow engine would be overkill.

Q: How do you guarantee idempotent consumers using a unique idempotency-key in your database?

A: Before processing a message/command, the consumer checks a dedicated table (or a unique constraint on an existing table) for that message's idempotency key (often the same as the message's unique event ID). If a row already exists for that key, the message has already been processed - skip reprocessing and return the previously recorded result/outcome. If not, process the message and insert the idempotency key row as part of the same transaction as the business logic's own writes, so a crash between 'processed the business logic' and 'recorded the idempotency key' can't happen - both succeed or both roll back together.

```

@Transactional
public void processPayment(PaymentEvent event) {
    if (idempotencyRepository.existsByEventId(event.getEventId())) {
        return; // already processed - safe to skip, this is what makes it idempotent
    }
    paymentService.charge(event.getAmount(), event.getAccountId());
}

```



```

    idempotencyRepository.save(new ProcessedEvent(event.getEventId(), Instant.now()));
    // both the charge and the idempotency record commit atomically in one transaction
}

```

Q: Resilience4j Circuit Breaker - how do you actually determine failureRateThreshold and slidingWindowSize?

A: failureRateThreshold (e.g., 50%) is the percentage of calls that must fail within the sliding window before the circuit trips to OPEN - set based on how tolerant the downstream dependency's normal failure rate is (a flaky-but-usually-fine dependency might warrant 50-60%; a critical dependency where any sustained failure is concerning might warrant a lower threshold like 30%). slidingWindowSize defines how many recent calls (count-based) or how much recent time (time-based) the failure rate is calculated over - too small and a brief blip trips the breaker unnecessarily; too large and the breaker reacts too slowly to a real ongoing outage. A common starting point is a count-based window of 20-100 calls, tuned based on the service's actual request volume.

Q: Walk through the Open -> Half-Open -> Closed state transition in detail.

A: CLOSED: calls pass through normally, and Resilience4j tracks the failure rate over the sliding window. Once failureRateThreshold is breached, the circuit trips to OPEN: all calls immediately fail-fast (calling the fallback directly) without even attempting the real downstream call, for a configured waitDurationInOpenState. After that wait elapses, the circuit moves to HALF_OPEN: a small, configured number of trial calls (permittedNumberOfCallsInHalfOpenState) are allowed through for real. If enough of those trial calls succeed (below the failure threshold again), the circuit closes fully, resuming normal traffic. If they still fail, it reopens and waits again before retrying.

Q: What's a good fallback mechanism for a circuit breaker, beyond just throwing an error?

A: Returning stale cached data (e.g., last-known-good product price/inventory count) is often better than failing outright for read-heavy, tolerant-of-staleness data - the user gets something useful instead of an error. For writes/critical operations that can't proceed without the real downstream call, sending the request to a Dead Letter Queue for later reprocessing (once the dependency recovers) preserves the request instead of losing it, while still letting the caller move on immediately rather than blocking.

```

@CircuitBreaker(name = "pricingService", fallbackMethod = "fallbackToCachedPrice")
public Price getLivePrice(String instrumentId) {
    return pricingClient.getPrice(instrumentId);
}

public Price fallbackToCachedPrice(String instrumentId, Throwable t) {
    return cache.getDefault(instrumentId, Price.unavailable(instrumentId));
}

```

11. Advanced Java Ecosystem & JVM Internals

Q: G1GC vs ZGC - explain the choice for a low-latency enterprise app on Java 17/21.

A: G1GC (Garbage-First) divides the heap into regions and tries to prioritize collecting regions with the most garbage first, aiming for a configurable pause-time target - it's been the default collector since Java 9 and works well for a broad range of applications, but pause times can still grow with heap size, especially during full GCs. ZGC is designed for very large heaps with pause times that stay under a few milliseconds regardless of heap size (multi-gigabyte to terabyte heaps), by doing almost all of its work concurrently with the application threads rather than stopping them. The trade-off: ZGC's concurrent work consumes more CPU/throughput overhead than G1GC for the same workload, so for a low-latency, SLA-sensitive service (e.g., a real-time pricing/trading path) ZGC's predictable sub-millisecond pauses are worth the throughput cost; for a throughput-oriented batch job where occasional longer pauses are acceptable, G1GC (or even the older Parallel collector) may be more efficient overall.

Q: How do Virtual Threads (Project Loom, Java 21) change how you write concurrent, I/O-bound code?

A: Before virtual threads, handling thousands of concurrent blocking I/O operations efficiently required either tying up thousands of expensive OS (platform) threads - which doesn't scale well past a few thousand - or rewriting the code in a reactive style (WebFlux/Reactor) to avoid blocking threads at all, at the cost of much harder-to-read, harder-to-debug code. Virtual threads are extremely lightweight, JVM-managed threads - you can create millions of them cheaply, and when a virtual thread blocks on I/O, the JVM automatically unmounts it from its underlying platform thread (freeing that platform thread to run other virtual threads), then remounts it once the I/O completes. This means you can write simple, traditional blocking-style code (like plain JDBC calls or RestTemplate calls) and still get reactive-like scalability for I/O-bound workloads, without the reactive programming complexity.

```
// Traditional platform thread pool - limited by OS thread overhead at high concurrency
ExecutorService platformPool = Executors.newFixedThreadPool(200);

// Virtual threads - can handle vastly more concurrent I/O-bound tasks cheaply
ExecutorService virtualPool = Executors.newVirtualThreadPerTaskExecutor();

virtualPool.submit(() -> {
    String result = restTemplate.getForObject(url, String.class); // blocking call - but cheap now
    process(result);
});
```

Q: What's an important caveat/pitfall with virtual threads that's worth mentioning if asked?

A: Virtual threads don't help (and can even hurt) for CPU-bound work - they're specifically designed to help when threads spend most of their time blocked on I/O, not doing computation. Also, a virtual thread that enters a synchronized block/method currently 'pins' to its underlying platform thread for the duration (can't unmount), which can reduce the scalability benefit if code has heavy use of synchronized around blocking calls - java.util.concurrent locks (ReentrantLock) don't have this pinning limitation and are preferred in virtual-thread-heavy code.

Q: Metaspace OOM vs Heap Space OOM - what's the difference, and what typically causes each?

A: Heap Space OOM ('java.lang.OutOfMemoryError: Java heap space') means the heap - where actual objects live - is full and the GC can't reclaim enough space; typically caused by a memory leak (objects that should be garbage-collected but are still reachable, e.g., unclosed resources or objects accumulating in a static collection), or simply an undersized heap for the actual workload. Metaspace OOM ('java.lang.OutOfMemoryError: Metaspace') means the memory used for class metadata (loaded class definitions themselves, not object instances) is exhausted - commonly caused by dynamically loading/generating huge numbers of classes at runtime (e.g., certain proxy-heavy frameworks, or repeated hot-redeployment in some app servers) without ever unloading the old ones.

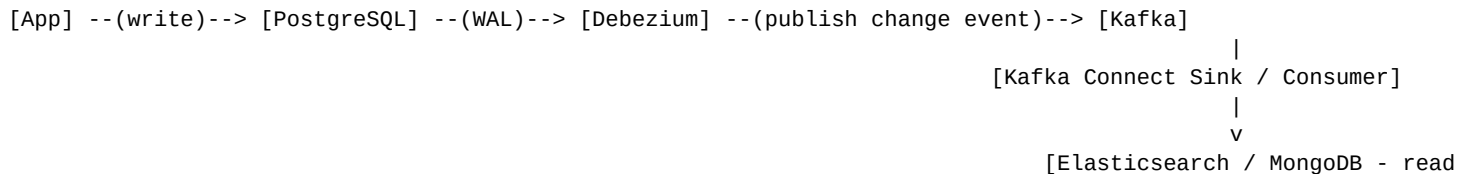
Q: How do you use Eclipse Memory Analyzer (MAT) or JProfiler to actually track down a memory leak - shallow vs retained heap?

A: Shallow heap is the memory an object itself occupies, not counting anything it references. Retained heap is the total memory that would be freed if that object became garbage-collectible - i.e., the object itself plus every other object that's only reachable through it. When hunting a leak, sort objects by retained heap (not shallow heap) in MAT's dominator tree view, since the real culprit is usually a single object (e.g., a static List, or a cache with no eviction) with a huge retained size, even if its own shallow size is tiny - it's holding onto a large chain of other objects. Common real-world causes this technique surfaces: unclosed database connections/ResultSets accumulating in a connection pool's internal tracking, ThreadLocal values never cleared in a pooled-thread environment (the thread is reused, but the ThreadLocal value silently persists across requests), or a long-lived static Map used as an ad-hoc cache with no size limit or TTL.

12. Enterprise Data Patterns & Caching Architecture

Q: CQRS + Event Sourcing: how do you keep a read-optimized store (Elasticsearch/Mongo) in sync with a write-optimized store (PostgreSQL)?

A: The write side (PostgreSQL) remains the transactional source of truth for commands/writes. Rather than the application dual-writing to both PostgreSQL and the read store directly (which risks the two falling out of sync if one write fails), a common pattern uses Change Data Capture via Debezium, which reads PostgreSQL's write-ahead log (WAL) directly and streams every committed change as an event onto Kafka, without the application needing to explicitly publish anything. A separate consumer (or Kafka Connect sink connector) reads those change events and updates the read-optimized store (Elasticsearch/Mongo) accordingly. This guarantees the read store eventually reflects every committed write, without the dual-write consistency risk of writing to two stores directly in application code.



Q: How do you handle eventual consistency lag when a user writes data but reads it back immediately (the classic CQRS gotcha)?

A: A few common mitigations: read-your-own-writes consistency - route a user's own subsequent reads (right after their own write) back to the write-side store or a synchronously-updated cache for a short window, rather than the eventually-consistent read replica, specifically for that user's session. Optimistic UI updates - update the UI immediately with the data the user just submitted (assume success), rather than waiting for a round-trip read from the lagging read store. Or simply surface the lag honestly where acceptable (e.g., 'your changes may take a few seconds to appear in search results') when strict read-your-writes isn't worth the added complexity for that particular feature.

Q: What is the dual-write problem, and how does the Transactional Outbox Pattern solve it?

A: The dual-write problem: if a service updates its database AND separately publishes a message/cache-invalidation event (e.g., to Kafka, or evicting a Redis key) as two separate operations, there's a window where one can succeed and the other fail - e.g., the DB commits but the process crashes before the Kafka publish, leaving downstream consumers (or the cache) with stale/incorrect data with no record that anything went wrong. The Transactional Outbox Pattern solves this by writing the event/message into an 'outbox' table in the SAME database transaction as the actual business data change - both succeed or both roll back together, guaranteed by the database's own transactional guarantee. A separate background process (or Debezium reading the outbox table's own WAL changes) then reliably publishes those outbox rows to Kafka/cache-invalidation asynchronously, with retry, decoupled from the original transaction.

```
@Transactional
public void updateProductPrice(Long productId, BigDecimal newPrice) {
    Product product = productRepository.findById(productId).orElseThrow();
    product.setPrice(newPrice);
    productRepository.save(product);

    // Written in the SAME transaction as the business update above
    outboxRepository.save(new OutboxEvent(
        "PRODUCT_PRICE_CHANGED", productId, newPrice, Instant.now()));
    // A separate poller/Debezium process publishes this row to Kafka later,
    // guaranteeing it's only ever published if the price update itself actually committed
}
```

Q: How do you prevent a cache stampede, and how do you configure @Cacheable TTLs carefully?

A: A cache stampede happens when a popular cache key expires and many concurrent requests simultaneously miss the cache and hammer the database at once trying to recompute the same value. Mitigations: add jitter to TTLs (randomize expiry slightly per key, e.g., 300s +/- 30s, so many keys don't expire at the exact same instant); use a lock/single-flight pattern so only one request recomputes a missing value while others wait for that result rather than all hitting the DB independently; or proactively refresh hot keys slightly before they expire (probabilistic early recomputation) instead of waiting for a hard expiry and a full cache miss.

```
@Cacheable(value = "products", key = "#id")
public Product getProduct(Long id) {
    return productRepository.findById(id).orElseThrow();
}

// application.yml - jittered TTL example for a Redis-backed cache
spring:
  cache:
    redis:
      time-to-live: 300000 # base 5 min; pair with per-key jitter logic in a custom CacheManager
                          # to avoid many keys expiring in the exact same instant
```

Q: @Cacheable vs @CacheEvict vs @CachePut - when do you use each?

A: @Cacheable checks the cache first and only calls the method on a miss, caching the result afterward - use for reads. @CacheEvict removes an entry (or clears the whole cache with allEntries=true) - use after a delete or when data becomes stale. @CachePut always executes the method and updates the cache with the result unconditionally (unlike @Cacheable, it doesn't skip execution on a hit) - use after an update, so the cache reflects the new value immediately rather than waiting for the next read to repopulate it.

13. Final Skim Before the Call

- Have your own architecture diagram ready to draw/describe fluently - this WILL be asked, per real EPAM reports.
- Heartbeat/lease-expiration in service registry, routing mechanics in API Gateway, and Config Server fetch flow - expect deep dives, not just definitions.
- Practice narrating the N+1 fix, the latency fix, and the SARK sidecar telemetry story out loud with real numbers.
- CAP theorem: pick CP vs AP with a real example each. PACELC extends it to the non-partition case (latency vs consistency).
- Consistent hashing solves the 'redistribute everything on scale change' problem that modulo hashing has.
- NoSQL types: key-value (cache), document (flexible schema), column-family (write-heavy time-series), graph (relationships).
- For any HLD question: clarify scale first, then components, then data model, then the one hardest trade-off - narrate this order out loud.
- Saga's real drawback: eventual consistency + compensating transactions are harder to debug - say this from experience, not just theory.
- Kafka: same key -> same partition -> guaranteed order for that entity. At-least-once delivery means consumers must be idempotent.
- WebFlux = event-loop, non-blocking, scales concurrency with few threads. Spring MVC = thread-per-request, simpler, blocks on I/O.
- Resilience4j: failureRateThreshold + slidingWindowSize define when OPEN trips; HALF_OPEN runs limited trial calls before fully closing again.

- Idempotent consumers: check an idempotency-key table in the SAME transaction as the business write, skip if already processed.
- ZGC = sub-millisecond pauses regardless of heap size, at a throughput cost. G1GC = solid default, pauses can grow with heap size.
- Virtual threads (Java 21) let you write simple blocking code that scales like reactive code for I/O-bound work - but avoid synchronized blocks (pinning) and don't help CPU-bound work.
- MAT/heap dumps: sort by RETAINED heap, not shallow heap, to find the real leak culprit (static caches, unclosed connections, stale ThreadLocals).
- Dual-write problem -> Transactional Outbox Pattern: write the event to an outbox table in the SAME DB transaction as the business change.
- Cache stampede prevention: jittered TTLs, single-flight locking, or proactive early refresh of hot keys before hard expiry.
- Be ready to whiteboard your project's component diagram and name at least one real NFR (latency/scale/availability) you designed for.
- For soft-skill questions, use STAR and end with a concrete outcome, not just 'and it worked out.'
- If a topic goes beyond your real hands-on depth, say so plainly and pivot to what you've actually done - it reads better than guessing.

This is the round where your real experience does the talking - you've got plenty of it. Go get it.

Addendum: Servlets, Webhooks, Strategy Pattern, JUnit Testing & Code Smell Review

Prepared for the repeat Technical Discussion round · 8th Aug 2026 · Hyderabad
Covers gaps from the previous session's feedback and new topics raised

A1. Servlets

Q: What is a Servlet, and how does it relate to Spring MVC/Spring Boot?

A: A Servlet is the foundational Java class for handling HTTP requests - it predates Spring, and Spring MVC is actually built on top of the Servlet API. A Servlet runs inside a Servlet Container (Tomcat, Jetty), which manages its lifecycle: `init()` runs once on first load, `service()/doGet()/doPost()` runs once per incoming request, and `destroy()` runs on shutdown.

```
public class HelloServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/plain");
        response.getWriter().write("Hello, " + request.getParameter("name"));
    }
}
```

Q: How does a request actually reach your `@RestController` method under the hood?

A: It hits the embedded Tomcat server, which routes it to `DispatcherServlet` - Spring MVC's front controller, which is itself just a Servlet, auto-registered by Spring Boot. `DispatcherServlet` asks Spring's `HandlerMapping` which controller method matches the request's path/method, then invokes it. This is also the underlying reason Spring MVC is a thread-per-request model - that comes directly from the Servlet API's synchronous `doGet()/doPost()` design, one thread occupied per request for its full duration.

Q: What is the difference between a Filter and a Servlet?

A: A Servlet handles one specific endpoint/path's business logic and produces the actual response. A Filter sits in the request/response chain before and/or after a Servlet, used for cross-cutting concerns applied broadly across many endpoints - logging every request, validating an auth token, compressing responses - without each Servlet needing to implement that logic itself. Spring Security's filter chain (from your earlier prep) is a direct real-world example of this Filter pattern.

A2. Webhooks

Q: What is a webhook, and how does it differ from polling?

A: A webhook is how one system pushes a notification to another system the moment something happens, instead of the second system repeatedly asking 'anything new yet?' (polling). You register a URL with a provider ahead of time; when the relevant event occurs on their side, they make an HTTP POST to your URL with the event data - you find out instantly with no wasted repeated requests and no polling delay.

```
@RestController
public class PaymentWebhookController {

    @PostMapping("/webhooks/payment-completed")
    public ResponseEntity<Void> handlePaymentWebhook(@RequestBody PaymentEvent event,
                                                    @RequestHeader("X-Signature") String signature) {
        if (!isValidSignature(event, signature)) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
        }
        orderService.markPaid(event.getOrderID()); // should be idempotent - see below
        return ResponseEntity.ok().build();
    }
}
```

Q: Why must a webhook endpoint verify a signature, and why must it be idempotent?

A: Because a webhook endpoint is a public URL, anyone could POST fake event data to it if there's no verification - so the receiver checks an HMAC signature (computed from a shared secret) sent in a header to confirm the request genuinely came from the expected provider. Idempotency matters because providers typically retry webhook delivery if they don't receive a fast 200 response - so the same event may arrive twice, and processing it a second time shouldn't double-charge, double-notify, or otherwise duplicate the side effect. This connects directly to the idempotency-key pattern already covered for Kafka consumers - the same principle, different transport.

A3. Code Design: If-Else Chains -> Strategy Pattern (Discount Example)

A: This is the exact question you were asked, worked through in full - including the follow-up depth an interviewer at Lead/Senior level is listening for.

The original code (the problem)

```
public class Discount {
    public double giveDiscount(User user, double price) {
        if (user.categoryType.equals("REGULAR")) {
            return price * 1.0;
        }
        if (user.categoryType.equals("PREMIUM")) {
            return price * 0.9;
        }
        // adding GOLD, SILVER, PLATINUM means adding more if-blocks here, forever
        return price;
    }
}
```

Q: What's actually wrong with this code, beyond 'it needs more if-blocks'?

A: Two separate problems, and both need to be named explicitly: (1) String literals for category comparison - typo-prone, no compile-time safety, a silently-wrong string like 'REGLUAR' just falls through with no discount and no error. (2) The deeper issue: this violates the Open-Closed Principle (a class should be open for extension, closed for modification) - every new category requires opening and editing giveDiscount() itself, instead of adding new behavior without touching existing, already-tested code.

Step 1 - fix the string literal problem with an Enum

```
public enum UserCategory {
    REGULAR, PREMIUM, GOLD, SILVER, PLATINUM
}
```

A: This alone fixes the typo/type-safety problem, but if giveDiscount() still does if (user.getCategory() == UserCategory.REGULAR) {...} else if (...GOLD) {...}, the OCP violation is still there - you've cleaned up the data representation but not eliminated the branching that has to grow forever. This is exactly the gap flagged in the actual interview feedback: enum-for-safety is correct but incomplete on its own.

Step 2 - eliminate the if-else entirely: let the enum carry behavior

```
public enum UserCategory {
    REGULAR(1.0), PREMIUM(0.9), GOLD(0.85), SILVER(0.95), PLATINUM(0.8);

    private final double multiplier;
    UserCategory(double multiplier) { this.multiplier = multiplier; }
}
```

```

        public double apply(double price) { return price * multiplier; }
    }

    public class Discount {
        public double giveDiscount(User user, double price) {
            return user.getCategory().apply(price);    // one line, zero if-else
        }
    }
}

```

A: Adding a new category is now a single enum entry, with zero changes to Discount.java's business logic - this is what actually satisfies the Open-Closed Principle, and is the core signal a Strategy-pattern-style question like this is testing.

Step 3 - where DB/config-driven values still matter (and matter MORE at senior level)

A: If discount percentages need to change at runtime without a code deploy (a business/marketing-driven change, not an engineering one), back the multiplier with a DB-backed lookup instead of hardcoding it in the enum - e.g., a discount_rules table loaded into a Map, cached in-memory or Redis with a refresh job. Saying this explicitly signals you're thinking about who owns the data (business/marketing vs engineering), which is a genuinely senior-level distinction beyond just 'clean code.'

Follow-up: 'what if discount logic gets more complex than a flat multiplier' (tiered, coupon stacking)?

A: At that point, a flat enum-with-a-number stops being enough, and this becomes a full Strategy Pattern with Spring dependency injection - each category gets its own strategy implementation class, and Spring auto-wires all of them into a Map keyed by type.

```

public interface DiscountStrategy {
    double apply(User user, double price);
}

@Component
public class GoldDiscountStrategy implements DiscountStrategy {
    public double apply(User user, double price) {
        double base = price * 0.85;
        // tiered/coupon-stacking logic can live here without touching other categories at all
        return base;
    }
}

@Service
public class Discount {
    private final Map<UserCategory, DiscountStrategy> strategies;

    // Spring auto-injects every DiscountStrategy bean into this map, keyed by a method on each bean
    public Discount(Map<UserCategory, DiscountStrategy> strategies) {
        this.strategies = strategies;
    }

    public double giveDiscount(User user, double price) {
        return strategies.get(user.getCategory()).apply(user, price);
    }
}

```

» TIP: If you get any 'redesign this if-else chain' question in the repeat round, walk through this exact progression out loud: name the OCP violation explicitly -> enum for type safety -> enum-carries-behavior to kill the if-else -> mention DB-backed values for runtime flexibility -> offer the full Strategy-interface version if they push on complexity. Naming

'Open-Closed Principle' and 'Strategy Pattern' by name early is what signals you know the pattern, not just a clean-code instinct.

A4. Writing JUnit Tests (Division Method Example)

Q: Write a method that divides two integers, and its JUnit tests.

```
public class Calculator {
    public int divide(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("Cannot divide by zero");
        }
        return a / b;
    }
}

class CalculatorTest {

    private final Calculator calculator = new Calculator();

    @Test
    void shouldDivideTwoPositiveNumbers() {
        assertEquals(5, calculator.divide(10, 2));
    }

    @Test
    void shouldReturnZeroWhenDividendIsSmallerThanDivisor() {
        assertEquals(0, calculator.divide(3, 5)); // integer division truncates
    }

    @Test
    void shouldHandleNegativeNumbers() {
        assertEquals(-5, calculator.divide(-10, 2));
        assertEquals(5, calculator.divide(-10, -2));
    }

    @Test
    void shouldThrowArithmeticExceptionWhenDividingByZero() {
        ArithmeticException ex = assertThrows(
            ArithmeticException.class,
            () -> calculator.divide(10, 0)
        );
        assertEquals("Cannot divide by zero", ex.getMessage());
    }
}
```

Q: What makes this a good/complete set of tests, if asked to justify it?

A: It covers the happy path (normal division), an edge case that's easy to get wrong (integer truncation when the dividend is smaller than the divisor, and negative number handling), and the exception path (division by zero) using `assertThrows` rather than a `try/catch` with a manual `fail()` call - `assertThrows` is the modern, cleaner JUnit 5 idiom and also lets you assert on the exception's message. If pushed further, mention boundary values too - e.g., dividing `Integer.MIN_VALUE` by `-1` actually overflows in Java (a genuinely obscure but real edge case worth naming if you want to show real depth, though not something you'd necessarily add a test for by default).

Q: What's the difference between `assertEquals`, `assertThrows`, and `assertTrue/assertFalse` - when do you use each?

A: `assertEquals(expected, actual)` - compares an expected value against what the method actually returned; the most common assertion for pure functions. `assertThrows(ExceptionType.class, executable)` - verifies a

specific exception is thrown by wrapping the call in a lambda, and lets you further assert on the exception object itself (message, cause). assertTrue/assertFalse - for boolean conditions that don't fit a clean equality check, e.g., asserting a list isEmpty() or a string startsWith(...).

Q: How would you structure tests using @BeforeEach and parameterized tests, for something like this?

```
class CalculatorTest {
    private Calculator calculator;

    @BeforeEach
    void setUp() {
        calculator = new Calculator();    // fresh instance before every test - avoids shared state between tests
    }

    @ParameterizedTest
    @CsvSource({
        "10, 2, 5",
        "9, 3, 3",
        "-10, 2, -5",
        "7, 2, 3"
    })
    void shouldDivideVariousInputsCorrectly(int a, int b, int expected) {
        assertEquals(expected, calculator.divide(a, b));
    }
}
```

A: @ParameterizedTest with @CsvSource lets you cover many input/output combinations without writing a near-identical test method for each one - worth mentioning if asked how you'd scale test coverage without duplicating test code.

A5. Code Smell Identification - 'What's Wrong With This Code' Style

A: These 'spot the problem' questions are common in design-focused rounds. Below are worked examples covering the two patterns you mentioned: magic string literals instead of constants, and tightly-coupled method calls within a class.

Example 1: Magic string literals instead of named constants

```
// BEFORE - what's wrong here?
public class OrderProcessor {
    public void processOrder(Order order) {
        if (order.getStatus().equals("PENDING")) {
            order.setStatus("PROCESSING");
        }
        if (order.getPaymentMethod().equals("CREDIT_CARD")) {
            applyCardSurcharge(order);
        }
    }
}
```

Q: What's wrong with this, and how would you fix it?

A: Raw string literals scattered through business logic: no compile-time safety (a typo like 'PENIDNG' silently fails to match, with no error anywhere), no single source of truth for valid values (nothing stops someone elsewhere in the codebase from setting an invalid status string), and poor discoverability (an IDE can't autocomplete or 'find usages' a raw string the way it can a named constant/enum). Fix: use an enum for a

closed, known set of values like order status or payment method - the same fix pattern as the Discount question above.

```
// AFTER
public enum OrderStatus { PENDING, PROCESSING, SHIPPED, CANCELLED }
public enum PaymentMethod { CREDIT_CARD, DEBIT_CARD, PAYPAL }

public class OrderProcessor {
    public void processOrder(Order order) {
        if (order.getStatus() == OrderStatus.PENDING) {
            order.setStatus(OrderStatus.PROCESSING);
        }
        if (order.getPaymentMethod() == PaymentMethod.CREDIT_CARD) {
            applyCardSurcharge(order);
        }
    }
}
```

A: If the value genuinely isn't a closed, known set (e.g., a config key, or a value that varies per environment), a named `public static final String` constant is the right fix instead of an enum - the key point to say out loud is 'closed, known set of values -> enum; a single reusable literal -> named constant,' since interviewers sometimes probe which one you'd pick and why.

Example 2: Tightly-coupled method calls inside a class

```
// BEFORE - what's wrong here?
public class Order {
    private List<Item> items;
    private Customer customer;

    public void checkout() {
        calculateTotal();
        applyDiscount();
        chargePayment();
        sendConfirmationEmail();
        updateInventory();
    }

    private void calculateTotal() { /* ... */ }
    private void applyDiscount() { /* ... */ }
    private void chargePayment() { /* directly calls a PaymentGateway singleton internally */ }
    private void sendConfirmationEmail() { /* directly calls an EmailService singleton internally */ }
    private void updateInventory() { /* directly calls an InventoryService singleton internally */ }
}
```

Q: What's wrong with this design, and how would you fix it?

A: The Order class is doing far too much - it's not just modeling order data, it's also orchestrating payment, email, and inventory concerns internally, each often reaching out to some global/static service reference. This violates the Single Responsibility Principle (a class should have one reason to change - now this class changes if payment logic changes, if email templates change, or if inventory logic changes) and makes it very hard to unit test `checkout()` in isolation, since you can't easily substitute a fake `PaymentGateway` or `EmailService` for a test without real payment/email side effects actually firing. It's also tightly coupled - Order directly knows about and depends on `PaymentGateway`, `EmailService`, and `InventoryService` concretely, rather than through injected abstractions.

```
// AFTER - responsibilities separated, dependencies injected via interfaces (Dependency Inversion)
public class Order {
    private List<Item> items;
    private Customer customer;
    // Order now only holds order data + its own domain logic (calculateTotal, applyDiscount)
}
```

```

    public double calculateTotal() { /* ... */ return 0.0; }
    public double applyDiscount(double total) { /* ... */ return total; }
}

@Service
public class CheckoutService {
    private final PaymentGateway paymentGateway;
    private final EmailService emailService;
    private final InventoryService inventoryService;

    // Constructor injection - dependencies are interfaces, easily mocked in tests
    public CheckoutService(PaymentGateway paymentGateway, EmailService emailService,
        InventoryService inventoryService) {
        this.paymentGateway = paymentGateway;
        this.emailService = emailService;
        this.inventoryService = inventoryService;
    }

    public void checkout(Order order) {
        double total = order.applyDiscount(order.calculateTotal());
        paymentGateway.charge(order.getCustomer(), total);
        emailService.sendConfirmation(order);
        inventoryService.reduceStock(order.getItems());
    }
}

```

A: Now Order is a focused domain object, CheckoutService orchestrates the workflow, and every external dependency is an injected interface - so a unit test for checkout() can pass in mock PaymentGateway/EmailService/InventoryService implementations and verify the orchestration logic without any real side effects. This is the same SRP + Dependency Inversion combination from your earlier SOLID prep, just applied as a 'spot the problem' exercise instead of a definition question.

Example 3: Depending on a concrete class instead of an interface (your own example)

```

// BEFORE - what's wrong here?
public class Order {
    @Autowired
    private PaymentService paymentService;    // concrete class, not an interface

    public void pay() {
        paymentService.pay();
    }
}

```

Q: What's wrong with this, and how would you fix it?

A: Order depends directly on a concrete PaymentService class rather than an abstraction - this violates the Dependency Inversion Principle (depend on abstractions, not concretions). Two concrete problems this causes: (1) you can't swap the payment implementation (e.g., add PayPal or Stripe alongside an existing card processor) without either modifying PaymentService itself or creating awkward conditional logic inside it - a fresh OCP violation on top of the DIP one. (2) It's harder to unit test Order.pay() in isolation, since Spring will try to wire a real PaymentService bean unless you specifically mock the concrete class, which is more brittle than mocking an interface.

```

// AFTER - introduce an interface, each payment type implements it
public interface PaymentService {
    void pay();
}

@Component("cardPayment")

```



```

public class CardPaymentService implements PaymentService {
    public void pay() { System.out.println("Paying via card..."); }
}

@Component("upiPayment")
public class UpiPaymentService implements PaymentService {
    public void pay() { System.out.println("Paying via UPI..."); }
}

public class Order {
    private final PaymentService paymentService;    // depends on the abstraction now

    // constructor injection - Spring supplies whichever implementation is wired/qualified
    public Order(@Qualifier("cardPayment") PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void pay() {
        paymentService.pay();
    }
}

```

A: Now Order.pay() works identically no matter which PaymentService implementation is injected - a new payment type (Stripe, PayPal, wallet) is a new class implementing the interface, with zero changes to Order itself. This is the exact same Strategy-pattern shape as the Discount example earlier: identify the varying behavior, pull it behind an interface, let each variant be its own implementation, and inject rather than hardcode. If the interviewer follows up with 'how would Order pick between card vs UPI at runtime, not just at wiring time,' that's the same Map Spring-injected-map pattern from the Discount follow-up - a factory/strategy-map resolves the right bean per request instead of one fixed @Qualifier per Order.

» TIP: For any 'what's wrong with this code' question, narrate in this order: (1) name the specific smell/principle violated by its proper name (OCP, SRP, tight coupling, magic literals), (2) explain the concrete pain it causes (untestable, error-prone, hard to extend) - not just 'it's not clean', (3) show the fixed version, (4) mention the trade-off if there is one (e.g., more classes/files for better separation). Naming the principle explicitly is consistently the difference between a good and a great answer in these rounds.

Addendum 2:

More System-Design-Flavored Code Smell Questions

Additional 'what's wrong with this code' exercises, senior/system-design level

A6. More Code Smell / 'What's Wrong With This' Exercises

A: These lean more system-design/scalability than the earlier pure-OOP examples - the kind of thing that surfaces in a live code review or architecture discussion, not just a textbook SOLID violation.

Example 4: Controller talking directly to the Repository (missing service layer)

```
// BEFORE - what's wrong here?
@RestController
@RequestMapping("/api/orders")
public class OrderController {

    @Autowired
    private OrderRepository orderRepository;

    @PostMapping
    public Order createOrder(@RequestBody Order order) {
        if (order.getItems().isEmpty()) {
            throw new IllegalArgumentException("Order must have items");
        }
        order.setStatus(OrderStatus.PENDING);
        order.setCreatedAt(Instant.now());
        return orderRepository.save(order);
    }
}
```

Q: What's wrong with this, and how would you fix it?

A: Business logic (validation, default status, timestamping) is sitting directly in the Controller, and the Controller talks straight to the Repository with no Service layer in between. This breaks separation of concerns: the Controller's job should be HTTP concerns only (request/response mapping, status codes), not business rules. Practical pain this causes: if the same order-creation logic is ever needed from another entry point (a scheduled job, an internal admin tool, a Kafka consumer reacting to an event), you either duplicate this logic or awkwardly call the Controller itself. It's also harder to unit test business rules in isolation without spinning up the whole web layer (MockMvc) just to test 'does an empty-items order get rejected.'

```
// AFTER - Controller -> Service -> Repository layering restored
@RestController
@RequestMapping("/api/orders")
public class OrderController {
    private final OrderService orderService;
    public OrderController(OrderService orderService) { this.orderService = orderService; }

    @PostMapping
    public ResponseEntity<Order> createOrder(@RequestBody OrderRequest request) {
        Order created = orderService.createOrder(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }
}

@Service
public class OrderService {
    private final OrderRepository orderRepository;
    public OrderService(OrderRepository orderRepository) { this.orderRepository = orderRepository; }

    public Order createOrder(OrderRequest request) {
        if (request.getItems().isEmpty()) {
            throw new IllegalArgumentException("Order must have items");
        }
        Order order = new Order(request.getItems(), OrderStatus.PENDING, Instant.now());
        return orderRepository.save(order);
    }
}
```

```
    }
}
```

Example 5: Swallowing exceptions silently

```
// BEFORE - what's wrong here?
public void syncInventory(String sku) {
    try {
        inventoryClient.sync(sku);
    } catch (Exception e) {
        // do nothing
    }
}
```

Q: What's wrong with this, and how would you fix it?

A: Catching a broad Exception and doing nothing with it hides real failures completely - if the inventory sync call is failing for every SKU due to, say, an expired auth token or a downstream outage, nobody finds out until someone notices stale inventory data days later, with zero trace of what went wrong or when it started. This is one of the most dangerous 'looks fine, isn't' smells - the code doesn't crash, so it looks healthy in a demo, but it's silently failing in production. Fix: log the failure with enough context to debug it, and decide deliberately whether the caller needs to know (rethrow/propagate) or whether a metric/alert should fire so someone notices the pattern even if this one call is allowed to fail gracefully.

```
// AFTER
public void syncInventory(String sku) {
    try {
        inventoryClient.sync(sku);
    } catch (InventorySyncException e) {
        log.error("Failed to sync inventory for SKU {}: {}", sku, e.getMessage(), e);
        meterRegistry.counter("inventory.sync.failures").increment();
        // deliberate choice: don't rethrow here since a single SKU failure shouldn't block a batch job,
        // but the log + metric make sure the failure is visible and alertable, not silent
    }
}
```

» TIP: Catching a specific exception type (not generic Exception) is itself part of the fix - a bare catch (Exception e) also swallows genuine programming bugs (NullPointerException, etc.) alongside the expected failure case, making real defects just as invisible as expected ones.

Example 6: Hardcoded configuration values inside business logic

```
// BEFORE - what's wrong here?
public class NotificationService {
    public void sendAlert(String message) {
        String apiUrl = "https://api.notify-provider.com/v1/send";
        String apiKey = "sk_live_4f9a8b7c6d5e4f3a2b1c0d"; // hardcoded secret!
        // ... call apiUrl with apiKey
    }
}
```

Q: What's wrong with this, beyond 'don't hardcode secrets'?

A: Two separate problems worth naming individually: (1) a hardcoded API key committed to source control is a genuine security vulnerability - anyone with repo access (or anyone who finds it in git history, even after deletion) has the live credential. (2) even ignoring the secret, hardcoding the URL means switching providers, environments (dev/staging/prod each need different endpoints), or rotating keys all require a code change and redeploy instead of a config change - this connects directly to the Config Server / externalized configuration material from earlier prep.

```
// AFTER
```

```

@Service
public class NotificationService {
    @Value("${notification.api.url}")
    private String apiUrl;

    // apiKey sourced from a secrets manager (Vault/AWS Secrets Manager) or env variable,
    // injected the same way - never committed to the config repo itself, let alone the code
    @Value("${notification.api.key}")
    private String apiKey;

    public void sendAlert(String message) {
        // ... call apiUrl with apiKey
    }
}

```

Example 7: Shared mutable state in a Spring singleton bean (thread-safety smell)

```

// BEFORE - what's wrong here?
@Service
public class ReportGenerator {
    private List<String> currentReportRows = new ArrayList<>(); // instance field, mutable

    public void addRow(String row) {
        currentReportRows.add(row);
    }

    public List<String> generateReport() {
        List<String> result = currentReportRows;
        currentReportRows = new ArrayList<>();
        return result;
    }
}

```

Q: What's wrong with this, and why does it only show up under load?

A: Spring beans are singletons by default - one shared instance serves every concurrent request. This class keeps per-request state (`currentReportRows`) as an instance field, so if two users call `addRow()/generateReport()` concurrently, their data interleaves and corrupts each other's report - classic race condition. This is a particularly dangerous smell because it works perfectly in local testing and low-traffic demos (one request at a time, no interleaving), and only manifests under real concurrent production load, making it hard to catch in code review unless you're specifically looking for shared mutable state in a singleton.

```

// AFTER - no shared mutable state; each call is self-contained and thread-safe
@Service
public class ReportGenerator {
    public List<String> generateReport(List<String> rows) {
        // rows is passed in and returned per-call, never stored as shared instance state
        return new ArrayList<>(rows);
    }
}

// If per-request accumulation is genuinely needed, build it up in the CALLER's local scope
// (a method-local List), not as a field on a shared singleton bean.

```

Example 8: N+1-style blocking calls to another microservice inside a loop

```

// BEFORE - what's wrong here?
public List<OrderDetail> getOrderDetails(List<Long> orderIds) {
    List<OrderDetail> details = new ArrayList<>();
    for (Long id : orderIds) {
        Order order = orderClient.getOrder(id); // one HTTP call per order...
    }
}

```

```

        Customer customer = customerClient.getCustomer(order.getCustomerId()); // ...plus one more per order
        details.add(new OrderDetail(order, customer));
    }
    return details;
}

```

Q: What's wrong with this, and how would you fix it, given it's calling other services rather than a database?

A: This is the N+1 problem's distributed-systems cousin - instead of N+1 extra DB queries, it's N*2 extra network round-trips to other microservices, which is even more expensive (network latency, and now two other services' availability/load are on the critical path for every single item). For 100 order IDs, that's 200 blocking HTTP calls made sequentially. Fix: batch the calls if the downstream services support a bulk endpoint (getOrders(List ids) instead of one-at-a-time), and/or make the calls concurrently instead of sequentially if a bulk endpoint isn't available.

```

// AFTER - batched calls instead of one-per-item
public List<OrderDetail> getOrderDetails(List<Long> orderIds) {
    List<Order> orders = orderClient.getOrders(orderIds); // ONE bulk call
    List<Long> customerIds = orders.stream().map(Order::getCustomerId).distinct().toList();
    Map<Long, Customer> customers = customerClient.getCustomers(customerIds) // ONE bulk call
        .stream().collect(Collectors.toMap(Customer::getId, c -> c));

    return orders.stream()
        .map(order -> new OrderDetail(order, customers.get(order.getCustomerId())))
        .collect(Collectors.toList());
}

```

» TIP: If a bulk endpoint genuinely doesn't exist and can't be added, the next-best fix is firing the per-item calls concurrently (e.g., via `CompletableFuture.allOf` or virtual threads from the earlier JVM prep) rather than sequentially - still N calls, but they overlap in time instead of stacking latency linearly.

Example 9: Exposing JPA entities directly through the REST API (missing DTO layer)

```

// BEFORE - what's wrong here?
@Entity
public class Employee {
    @Id Long id;
    String name;
    String ssn; // sensitive - social security number
    Double internalRating; // internal-only performance data
    @ManyToOne Department department;
}

@RestController
public class EmployeeController {
    @GetMapping("/api/employees/{id}")
    public Employee getEmployee(@PathVariable Long id) {
        return employeeRepository.findById(id).orElseThrow(); // returns the ENTITY directly
    }
}

```

Q: What's wrong with returning the entity directly from the controller?

A: Whatever fields exist on the JPA entity get serialized straight into the API response - including sensitive fields (ssn) or internal-only fields (internalRating) that should never be exposed externally, and this is easy to miss since it's not obvious from the controller code alone what the entity actually contains. It also tightly couples your public API contract to your internal database schema - a column rename or a new lazy-loaded relationship on the entity can silently change or break the API response (or trigger a `LazyInitializationException` if a lazy

association gets serialized outside a transaction), even though nobody touched the controller.

```
// AFTER - explicit DTO controls exactly what's exposed
public record EmployeeDto(Long id, String name, String departmentName) {}

@RestController
public class EmployeeController {
    @GetMapping("/api/employees/{id}")
    public EmployeeDto getEmployee(@PathVariable Long id) {
        Employee employee = employeeRepository.findById(id).orElseThrow();
        return new EmployeeDto(employee.getId(), employee.getName(), employee.getDepartment().getName());
    }
}
```

Example 10: A write endpoint with no idempotency protection

```
// BEFORE - what's wrong here?
@PostMapping("/api/orders")
public Order createOrder(@RequestBody OrderRequest request) {
    Order order = new Order(request.getItems(), request.getCustomerId());
    return orderRepository.save(order); // no duplicate check at all
}
```

Q: What's wrong with this, and when does it actually bite in production?

A: If the client retries the request - due to a network timeout where the request actually succeeded server-side but the response was lost, or a user double-clicking 'submit' - this creates two separate orders for what the user intended as one action. This connects directly to the idempotency-key pattern already covered for Kafka consumers and webhooks - the same principle applies to any write-triggering API endpoint, not just async message consumers.

```
// AFTER - client-supplied idempotency key, checked before creating
@PostMapping("/api/orders")
public ResponseEntity<Order> createOrder(@RequestBody OrderRequest request,
                                         @RequestHeader("Idempotency-Key") String idempotencyKey) {
    Optional<Order> existing = orderRepository.findByIdempotencyKey(idempotencyKey);
    if (existing.isPresent()) {
        return ResponseEntity.ok(existing.get()); // safe to return the same result again
    }
    Order order = new Order(request.getItems(), request.getCustomerId(), idempotencyKey);
    return ResponseEntity.status(HttpStatus.CREATED).body(orderRepository.save(order));
}
```

» TIP: Across all of these: the strongest answer format is still the same four-step narration - name the smell/principle explicitly, explain the concrete production pain (not just 'it's not clean'), show the fix, and flag the trade-off if there is one. For system-design-flavored smells specifically, the 'concrete pain' step is often about scale/concurrency/security rather than pure readability - naming which one it is shows you're reasoning like someone who's actually operated this kind of system, not just recognizing a textbook pattern.